

MOUSSE: Multi-Optimizer Unified Scaling Strategy for Efficiency in GPT-2 Fine-Tuning

Stanford CS224N Default Project

William Ekberg
ICME
Stanford University
ekberg@stanford.edu

Eden Luvishis
ICME
Stanford University
edenl@stanford.edu

Veronica Yakubovich
ICME
Stanford University
vyakubov@stanford.edu

Abstract

This project builds upon the default task of building and fine-tuning GPT-2. The primary goal is to investigate whether second-order preconditioned optimizers, specifically Shampoo and SOAP, can improve training efficiency and convergence speed compared to the standard Adam optimizer. The project consists of two main components: first, implementing key components of GPT-2 and evaluating its capabilities on sentiment analysis, paraphrase detection, and text generation tasks; second, evaluating the impact of different optimization methods on fine-tuning efficiency. Fine-tuning large language models is computationally expensive; thus, identifying optimization techniques that reduce the number of required iterations could lower training costs and enhance scalability for NLP applications. Shampoo and SOAP have demonstrated promising results in various deep learning settings, motivating an exploration of their effectiveness in transformer-based NLP fine-tuning. To evaluate their effectiveness, we compare Shampoo, SOAP, SGD, and Adam when fine-tuning GPT-2, measuring convergence rates, computational efficiency, and final loss. Additionally, we propose and evaluate hybrid optimization approaches, where different GPT-2 layers utilize different optimizers tailored to their specific characteristics.

1 Key Information to include

Mentor, External Collaborators, Sharing Project: N/A
Using 3 late days shared between team members

2 Introduction

Fine-tuning Large Language Models requires significant computational power and resources, which makes optimization efficiency an important research area. Standard first-order optimizers like AdamW have proven effective for training transformers, but they often struggle with slow convergence and sensitivity to hyperparameters. Meanwhile, second-order preconditioning methods, such as Shampoo and SOAP, offer theoretical improvements by incorporating curvature information, yet their scalability to large models remains an open question.

In this work, we investigate the impact of different optimization strategies on fine-tuning GPT-2. Specifically, we explore whether second-order preconditioning methods, such as Shampoo and SOAP, can improve training efficiency and convergence speed compared to AdamW. Additionally, we introduce MOUSSE (Multi-Optimizer Unified Scaling Strategy for Efficiency), a novel hybrid approach that assigns different optimizers to different layers within GPT-2, leveraging their unique optimization properties.

We evaluate these optimization strategies on two tasks: paraphrase detection using the Quora dataset and sonnet generation using a corpus of Shakespearean sonnets. Our experiments compare training stability, convergence speed, and final model performance for AdamW, SGD, SOAP, Shampoo, and MOUSSE. Given the computational limitations of second-order methods, we analyze their practical feasibility and explore whether a structured multi-optimizer approach can make large-scale NLP models more efficient without sacrificing performance.

3 Related Work

Our work explores second-order preconditioning methods in the context of transformer fine-tuning. This aligns with existing research on adaptive optimization methods but extends previous approaches by incorporating layer-wise adaptive preconditioning.

Second-order optimization methods, such as Shampoo [1] and Kronecker-Factored Approximate Curvature (K-FAC) [2], improve gradient-based optimization by estimating curvature information to precondition updates. Shampoo maintains full matrix gradient covariances to derive preconditioners, leading to faster convergence in deep learning tasks. However, Shampoo’s computational and memory overhead remains a limiting factor in large-scale models. K-FAC uses Kronecker factorization to approximate the Fisher information matrix, enabling more efficient second-order updates, though it also faces scalability challenges.

Recent advances have sought to improve the applicability of Shampoo to LLMs. SOAP [3] builds on Shampoo by integrating AdamW-style updates in the preconditioner eigenbasis. This modification maintains the benefits of Shampoo’s second-order updates while improving robustness to hyperparameter selection and reducing memory overhead. SOAP has also shown better performance over AdamW and Shampoo in language modeling tasks, particularly in terms of convergence efficiency and loss reduction [3].

Hybrid approaches, such as SWATS [4] and MAS (Mixing ADAM and SGD) [5], attempt to combine first-order methods like Adam with momentum-based SGD to balance stability and generalization. MAS optimizes by dynamically weighting Adam and SGD contributions, demonstrating superior performance in vision and NLP tasks. Our work draws inspiration from these hybrid strategies, exploring how different GPT-2 layers benefit from distinct optimizers.

Building on the effectiveness of Shampoo, SOAP, and hybrid optimization approaches individually, our contribution integrates these methods into a unified framework tailored for transformer fine-tuning. By adopting a layer-wise strategy, we aim to improve training efficiency while maintaining robust generalization, addressing the computational constraints of large-scale NLP models.

4 Approach

In this section, we detail our methodological approach to improving optimization in deep neural networks through second-order preconditioning methods. Specifically, we investigate various optimization strategies including Second-Order Approximate Preconditioning (SOAP) [3], Shampoo [1], and Kronecker-Factored Approximate Curvature (KFAC) [2]. Additionally, we introduce a novel multi-layer optimization structure, MOUSSE, which leverages different optimization algorithms for specific layer groups within the network.

Our baseline optimizers include widely recognized second-order methods alongside standard first-order optimization. SOAP provides an efficient second-order approximation by using simplified curvature estimates, offering robustness against gradient noise while maintaining reasonable computational and memory complexity. SOAP estimates a local curvature matrix at each layer independently and updates parameters using the preconditioned gradient given by [3]:

$$W^{(l)} \leftarrow W^{(l)} - \eta H_{\text{SOAP}}^{(l)-1} \nabla_{W^{(l)}} J(W), \quad (1)$$

where $H_{\text{SOAP}}^{(l)}$ represents SOAP’s approximate curvature matrix at layer l , η is the learning rate, and $\nabla_{W^{(l)}} J(W)$ denotes the gradient of the loss J with respect to the layer’s parameters, present in all gradient-based optimizers.

We also consider Shampoo as a baseline method, which employs second-order approximations by maintaining full matrix gradient covariances. Shampoo computes and updates a preconditioner using these covariance matrices, as follows:

$$G_t^{(l)} = G_{t-1}^{(l)} + \nabla_{W^{(l)}} J(W_t) \nabla_{W^{(l)}} J(W_t)^\top, \quad (2)$$

and the parameter update is performed by applying a preconditioner derived from matrix root approximations:

$$W_{t+1}^{(l)} \leftarrow W_t^{(l)} - \eta G_t^{(l)-\frac{1}{2}} \nabla_{W^{(l)}} J(W_t). \quad (3)$$

However, in practice, Shampoo posed substantial computational challenges, frequently encountering out-of-memory errors on both GPU and CPU due to its intensive memory requirements associated with performing full preconditioning matrix decompositions. Shampoo was initially developed for simpler neural networks, with much fewer trainable parameters than the 50000 we encounter in even the smallest GPT-2 model. As a result, we include Shampoo primarily as a conceptual baseline and note its practical limitations in our experiments.

We briefly explored Kronecker-Factored Approximate Curvature (KFAC), a powerful second-order method known for its factorization of the Fisher Information Matrix (FIM) into Kronecker products, providing

efficient approximations for large neural networks [2]. KFAC updates parameters using these factorizations to precondition gradients effectively, generally resulting in accelerated convergence. While promising in theory, our implementation of KFAC encountered similar issues to Shampoo because of the extremely large preconditioning matrices.

Motivated by these observations, we developed a novel optimization framework called MOUSSE (Multi-Optimizer Unified Scaling Strategy for Efficiency), aimed at strategically employing multiple optimizers tailored to distinct layers within deep neural networks. The key intuition underlying MOUSSE is that different components of a neural network architecture exhibit varying optimization characteristics and curvature dynamics, suggesting the benefit of specialized optimization strategies per layer group. Our final implementation of Mousse applies the following structured multi-optimizer scheme:

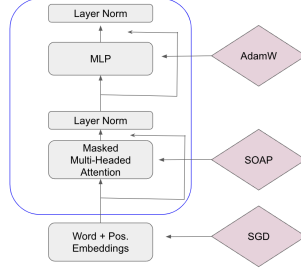


Figure 1: MOUSSE Diagram

For a neural network with L total layers, we partition the layers based on their functional roles into three groups, assigning an optimizer best suited to the characteristics of each group:

1. *Embedding Layers*: Due to their stability and relatively simple optimization landscape, these layers are optimized using standard Stochastic Gradient Descent (SGD) with momentum. SGD is fast, and efficiently find the optimal weights for simpler curvatures. The update rule for these embedding layers is given by:

$$W_{t+1}^{(\text{emb})} \leftarrow W_t^{(\text{emb})} - \eta \nabla_{W^{(\text{emb})}} J(W_t) + \mu (W_t^{(\text{emb})} - W_{t-1}^{(\text{emb})}), \quad (4)$$

where μ represents the momentum term.

2. *Attention Layers*: These layers have highly structured curvature patterns benefiting from second-order approximations, making SOAP an optimal choice. Attention layer parameters thus follow the SOAP update:

$$W_{t+1}^{(\text{attn})} \leftarrow W_t^{(\text{attn})} - \eta H_{\text{SOAP}}^{(\text{attn})-1} \nabla_{W^{(\text{attn})}} J(W_t). \quad (5)$$

3. *Feed-Forward Layers*: These layers often exhibit less smooth optimization dynamics, and therefore we utilize our implementation of AdamW, an adaptive first-order optimizer known for its robust handling of noisy gradients and suitability for fine-grained parameter updates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{W^{(\text{ff})}} J(W_t), \quad (6)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{W^{(\text{ff})}} J(W_t))^2, \quad (7)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad (8)$$

$$W_{t+1}^{(\text{ff})} \leftarrow W_t^{(\text{ff})} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda W_t^{(\text{ff})}, \quad (9)$$

where m_t and v_t represent the first and second moment estimates, β_1 and β_2 are AdamW hyperparameters, and λ controls weight decay.

The combination of these optimizers within the MOUSSE framework represents our primary original contribution. This structured multi-optimizer setup explicitly leverages distinct curvature properties and optimization challenges across different neural network layers, facilitating improved convergence stability, computational efficiency, and model performance relative to single-optimizer baselines. We implemented the Mousse framework and conducted extensive hyperparameter tuning ourselves, clearly differentiating our original work from publicly available implementations. For AdamW, we used our own implementation from the base default project, whereas SOAP and Shampoo were taken from existing implementations [6] [7]. In addition, we used the PyTorch Optimizer version of SGD [11].

In summary, our methodological approach introduces MOUSSE, a novel multi-layer optimization strategy integrating SGD for embedding layers, SOAP for attention layers, and AdamW for feed-forward layers. We consider SOAP, Shampoo, and briefly KFAC as conceptual baselines, with AdamW, SOAP, and SGD acting as our final baseline optimizer choices. Our experimental results (Section 5.4) demonstrate that MOUSSE successfully addresses many of the optimization challenges encountered by these baselines, leading to more stable training dynamics and faster convergence.

5 Experiments

This section contains the following.

5.1 Data

The data sources we used to train and evaluate our GPT-2-based model covered classification tasks and generative tasks.

5.1.1 Paraphrase Detection: Quora Dataset

This dataset from Quora had question pairs labelled paraphrase or non-paraphrase [10]. It was split up into a training dataset with 283,011 examples, a dev dataset of 40,430, and a test dataset of 80,860.

The model takes both sentences and returns a binary classifier output ("yes" or "no") based on if it is a paraphrase or not.

Table 1: Description and example of paraphrase detection

Field	Example
id	82861 44ae9297-536f-47d3-bc3a-58c7f874c3fc
sentence1	How do I start learning about artificial intelligence?
sentence2	How do you learn artificial intelligence?
Result: is_duplicate (1 or 0)	"Yes" → 1

5.1.2 Sonnet Dataset

This dataset has 131 Shakespearean sonnets given by the default project, each with 14 lines. This was split up into a 121 sonnet train dataset, 10 sonnet validation set used for early stopping, and then a head out dataset of just the first three lines of the sonnets.

For the generation task, these first three lines of a sonnet were provided as a prompt. The model’s objective was to generate the remaining 11 lines.

THE FIRST THREE INPUT LINES:

"What potions have I drunk of siren tears Distilled from limbeck's fowl as hell within, Applying fears to hopes and hopes to fears,"

THE CONTINUED OUTPUT:

"Still losing when I saw myself to win! What wretched errors hath my heart committed, Whilst it hath thought itself so blessed never! How have mine eyes out of their spheres been fitted In the distraction of this madding fever! O, benefit of ill! Now I find true That better is by evil still made better; And ruined love, when it is built anew, Grows fairer than at first, more strong, far greater. So I return rebuked to my content, And gain by ills thrice more than I have spent."

5.2 Evaluation method

We evaluate our optimization methods quantitatively and qualitatively across two NLP tasks: paraphrase detection and sonnet generation.

Quantitative Metrics

- **Paraphrase Detection**

- **Accuracy:** Defined as the percentage of correctly classified sentence pairs on the held-out dev dataset and measures the model’s classification performance.

- **Cross-Entropy Loss:** Measures how well the predicted probability distribution aligns with the ground truth labels. Lower loss indicates higher confidence and better-calibrated predictions.
- **Sonnet Generation**
 - **ChrF Score (Character-level F-score):** Measures the character-level overlap between the model-generated text and a set of reference sonnets.
- **Computational Efficiency and Training Stability**
 - **Training Time per Epoch:** We record the total time (in minutes) required to complete training. Lower training times per epoch indicate better computational efficiency.
 - **Loss Variance:** To quantify training stability, we measure the variance of the training loss across iterations. Lower variance in the loss curve indicates more stable convergence behavior.

Additionally, we qualitatively examine generated text from the sonnet generation task by manually inspecting generated sonnets for coherence, poetic fluency, and creativity, comparing outputs across different optimizers.

5.3 Experimental details

- Will All experiments were conducted on a GPU-equipped environment (NVIDIA T4 for all but SGD that was trained on an NVIDIA A100). The two tasks of 1: sonnet generation and 2: paraphrase detection used GPT-2 as the base model and finetuning with the datasets with specific parameters.

Table 2 and 3 summarizes the principal settings for each task.

Table 2: Optimizer hyperparameters for sonnet generation.

Parameter	AdamW	SOAP	Shampoo	SGD
Learning Rate	1×10^{-5}	1×10^{-5}	Default	1×10^{-5}
Betas	Default	(0.95, 0.95)	N/A	N/A
Weight Decay	Default	0.01	N/A	Default
Precondition Freq.	N/A	10	N/A	N/A

Table 3: Optimizer hyperparameters for paraphrase detection.

Parameter	AdamW	SOAP	Shampoo	SGD
Learning Rate	1×10^{-5}	1×10^{-5}	Default	1×10^{-5}
Betas	Default	(0.95, 0.95)	N/A	N/A
Weight Decay	0	0.01	N/A	Default
Precondition Freq.	N/A	10	N/A	N/A

Both tasks are configured to run for 5 epochs, with a default batch size of 8. For paraphrase detection, note that the help string suggests that batch sizes of 64 for SST and 8 for CFIMDB could be used to fit a 12GB GPU. PyTorch’s gradient scaling was used for GPU efficiency. For each epoch, the best-performing checkpoint (measured by validation loss for sonnet generation or validation accuracy for paraphrase detection) was saved and used for final evaluation.

Memory complications with Shampoo:

One key extension we explored was using the Shampoo optimizer in place of Adam. Shampoo is a second-order method designed to precondition the gradient using the statistics of each dimension. The optimizer keeps track of gradient covariance matrices and inverts them to obtain an improved preconditioned update. SVD for Matrix Inversion Under the hood, Shampoo needs to invert large matrices. One per tensor dimension is used to apply second-order updates. A common approach is to perform an eigendecomposition or SVD on each covariance matrix. Once the eigenvalues and eigenvectors are obtained, the matrix can be raised to a power and used as a preconditioner.

In practice, this can be problematic when those matrices become very large:

In GPT-2, embedding matrices can reach 50254×50254. Performing a full SVD on matrices of that size requires enormous memory and computation time. Moreover, PyTorch’s SVD (`torch.svd`) and `torch.linalg.svd` calls for such large matrices risk out-of-memory (OOM) errors or library-internal failures if the matrix or the internal workspace grows too big for GPU and CPU memory.

When implementing and running the Shampoo algorithm these issues came up and running the SVD calculations on the CPU and on the GPU resulted both in memory issues. This was tried on an A100 GPU with SXM4 40 GB and tested on a 30-core CPU with 214,7 GB of RAM.

Some mitigations were attempted but ultimately failed. The attempts for reducing the memory load were to try diagonal approximation, use a size threshold and try calculating the SVD both on the GPU and the CPU. Unfortunately, these attempts resulted in the same issues.

5.4 Results

Table 4: Paraphrase Detection Results

Optimizer	GPU	Train Time (5 epch)	Loss Variance	Dev Accuracy	Final Loss
AdamW	T4	536.55 min	0.04	89.7	0.015
SOAP	T4	3 days	0.064	89.2	0.01
Shampoo	A100	-	-	-	-
SGD	A100	121.25 min	0.028	73.9	0.39
MOUSSE	T4	904.30 min	0.04	89	0.008

Table 5: Sonnet Generation Results

Optimizer	GPU	Train Time (10 epchs)	Loss Variance	ChrF Score (Dev)	Final Loss
AdamW	T4	6.60 min	0.10	40.0	4.17
SOAP	T4	5.84 min	0.043	40.0	4.68
Shampoo	A100	-	-	-	-
SGD	A100	2.19 min	0.017	40.0	4.87
MOUSSE	T4	6.03 min	0.09	39.01	4.58

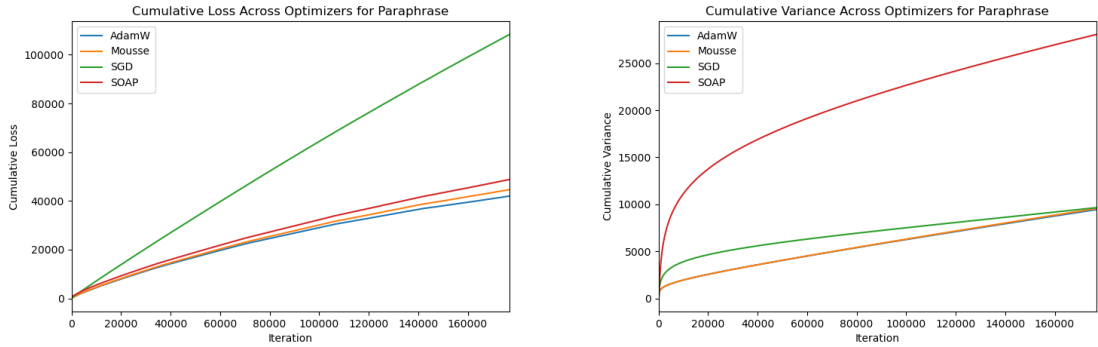


Figure 2: Comparison of Cumulative Loss and Variance Across Optimizers for Paraphrase

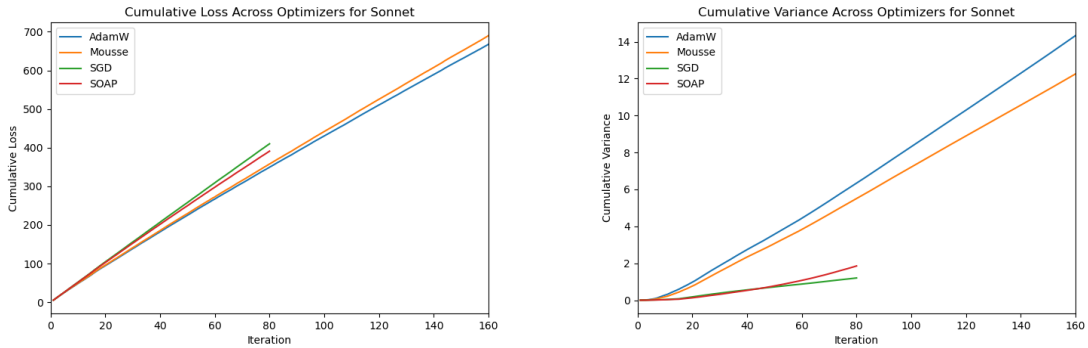


Figure 3: Comparison of Cumulative Loss and Variance Across Optimizers for Sonnet

Paraphrase detection: From table 4, AdamW achieved the highest Dev accuracy of 89.7% in about 536 minutes of training on a T4 GPU, with a final loss of 0.015. SOAP reached a Dev accuracy of 89.2% but required about three days on the same GPU. SGD finished with 121 minutes on an A100 and yielded the lowest accuracy of 73.9% and a relatively large final loss of 0.39. MOUSSE took about 904 minutes yet performed with 89% accuracy and reached the lowest final loss of 0.008.

In the loss and variance plots from figure 2, AdamW and MOUSSE showed relatively low cumulative loss overall. SGD converged more slowly, and SOAP’s variance was slightly higher. In terms of variance MOUSSE had the lowest variance over the iterations.

Sonnet generation: From table 5 one can see that all methods except MOUSSE produced similar Dev chrF scores around 40.0. MOUSSE’s chrF was slightly lower at 39.01. AdamW achieved the best final loss of 4.17, with SOAP (4.68) and MOUSSE (4.58) having a slightly higher loss. SGD trained the fastest with 2.19 minutes but on an A100 however it had the largest final loss of 4.87.

From figure 3, MOUSSE tracked close to AdamW but ended with a marginally higher loss. SGD and SOAP had lower variance but showed a higher loss.

Expectation vs Results

Our initial expectation was that second-order approaches Shampoo and SOAP would converge more quickly or yield better final losses than Adam-based methods thanks to their richer curvature information. Although SOAP did reduce the number of required updates in some cases, its large memory overhead prolonged overall wall-clock time more than anticipated. Shampoos memory constraints were also more problematic than expected.

On the other hand, we also expected MOUSSE to efficiently combine the advantages of second-order and first-order optimizers. It stabilized training variance and while maintaining similar losses as Adam. But this was at a higher computational and time cost. Thus, while second-order preconditioning did help in reducing variance and achieving strong final losses.

6 Analysis

To supplement the quantitative evaluation, we analyze various qualitative aspects of the optimizers’ performance. Specifically, we examine cumulative loss and variance trends across optimizers, a qualitative assessment of the generated sonnets, and the impact of optimization strategies on training dynamics.

Loss and Variance Across Optimizers

Paraphrase:

- Loss analysis: MOUSSE and AdamW exhibited the lowest cumulative loss, following nearly identical trajectories throughout training. In contrast, SGD had the highest cumulative loss, indicating poorer overall performance.
- Variance analysis: MOUSSE had the lowest cumulative variance among all optimizers, suggesting that it stabilized training dynamics effectively. AdamW had slightly higher variance, while SGD exhibited moderate variance but struggled with convergence.

Sonnet

- Loss analysis: Similar to paraphrase detection, SGD and SOAP maintained significantly higher cumulative loss throughout training, suggesting inefficient optimization. MOUSSE closely mirrored AdamW, indicating that it effectively leveraged multiple optimizers while maintaining stable training.
- Variance analysis: AdamW exhibited the highest variance among all optimizers, while MOUSSE’s variance was lower, which is a positive indicator of stable optimization. SOAP and SGD showed lower variance but at the cost of poor loss reduction, likely due to premature termination due to overfitting.

Generated Sonnet Output Analysis

To assess the quality of the generated sonnets, we manually analyzed the output produced by each optimizer. Our qualitative ranking of output quality was: MOUSSE > Adam > SOAP > SGD

- MOUSSE: Produced well-structured and coherent sonnets with correct spacing and line length. The generated output generally maintained a Shakespearian tone, though an occasional extra parentheses was observed.
- AdamW: Outputted reasonably fluent text but introduced some grammatical mistakes, such as unclosed parentheses and inconsistent line spacing.

- SOAP: Struggled with formatting issues, producing lines that were too long and included many unnecessary parentheses and spaces.
- SGD: Generated the least coherent output, often adding numbers at the end of lines, having excessively long lines, and modern-sounding phrases that significantly deviated from Shakespeare’s style. Some outputs even contained unrelated words like "[Scripture 129h]" in an unnatural context.

This indicates that MOUSSE provided the best balance between structure and stylistic accuracy, outperforming other optimizers in maintaining fluency and coherence even while having a slightly lower ChrF score.

Optimization Trade-offs and Practical Feasibility While second-order pre-conditioners like Shampoo and SOAP offer theoretical advantages, their practical usability is limited by high computational costs. Shampoo had extreme memory constraints, which rendered it infeasible for large-scale fine-tuning. SOAP, while computationally viable, showed the highest variance in paraphrase detection, suggesting inconsistent training stability.

MOUSSE successfully mitigated these challenges by applying a structured, layer-wise optimization strategy. By assigning SOAP to attention layers, AdamW to feedforward layers, and SGD to embeddings, MOUSSE retained the benefits of second-order optimization while maintaining computational efficiency.

7 Conclusion

In this project, we studied whether second-order preconditioning (Shampoo, SOAP) can improve GPT-2 fine-tuning compared to standard AdamW. We tested two tasks: paraphrase detection, and Shakespearean sonnet generation. We also introduced MOUSSE that applies SOAP to attention layers, AdamW to feed-forward layers, and SGD to embeddings.

Some limitations of this project are that we used different GPUs (T4 vs. A100) across our experiments. This makes direct comparisons of training times less consistent for SGD. Additionally, we varied the number of epochs for the sonnet generation task, meaning that more insights could be gained from having SGD and SOAP run be compared with AdamW and MOUSSE at iteration 160 in the plots.

In summary, using second-order optimization in LLMs remains challenging due to high memory and computational costs. On the other hand partitioning the model and carefully assigning different optimizers can bring the potential of getting second-order benefits while containing resource usage. We found that even pre-trained LLMs like GPT2 can benefit from targeted fine-tuning. For further exploration, it would be beneficial to explore more cost and memory-efficient second-order updates or adapter-based methods. Other directions include investigating dynamic optimizers to handle GPT-scale models better. Finally, MOUSSE proved to be an interesting approach that should be further explored with different combinations of optimizers.

Team contributions

All team members contributed equally to the project. Veronica developed and trained the SOAP model and added code to keep track of evaluation metrics, Eden built the main MOUSSE structure of the code as well as the base GPT-2 model, and Will trained and fine-tuned SGD and MOUSSE models.

References

- [1] Gupta, V., Koren, T., & Singer, Y. (2018). *Shampoo: Preconditioned stochastic tensor optimization*. International Conference on Machine Learning (ICML), PMLR.
- [2] Martens, J., Grosse, R. (2015) *Optimizing neural networks with Kronecker-factored approximate curvature*.
- [3] Vyas, N., Morwani, D., Zhao, R., Shapira, I., Brandfonbrener, D., Janson, L., & Kakade, S. (2024). *SOAP: Improving and stabilizing Shampoo using Adam*. arXiv preprint.
- [4] Keskar, N. S., & Socher, R. (2017). *Improving Generalization Performance by Switching from Adam to SGD*.
- [5] Landro, N., Gori, M., Marra, G., & Rodolà, E. (2017) *Mixing ADAM and SGD: A Combined Optimization Method*.
- [6] SOAP Implementation
- [7] Shampoo Implementation
- [8] Stanford Sentiment Treebank Dataset
- [9] Stanford Parser
- [10] Quora Dataset
- [11] Mykola Novik. *torch-optimizer – Collection of Optimization Algorithms for PyTorch*. Version 1.0.1, January 2020.